AMRITA VISHWA VIDYAPEETHAM

Coimbatore

PROJECT REPORT

Constraint-Aware Codon Optimization Using Greedy, MILP, Genetic Algorithms and NSGA-II

Submitted by

Name	Register Number
Vinith	CB.AI.U4CPS25055
Sanjai	CB.AI.U4CPS25043
Geethika	CB.AI.U4CPS25017
Advaith	CB.AI.U4CPS25004

School of Artificial Intelligence

Amrita Vishwa Vidyapeetham

Abstract

Codon optimization is a computational method for selecting synonymous codons in a coding DNA sequence to better match the codon-usage preferences of a target host while preserving the encoded amino-acid sequence. This project studies codon optimization for *Escherichia coli* K-12 MG1655 as a constrained optimization problem. Candidate DNA sequences are evaluated using Codon Adaptation Index (CAI), GC-content properties, edit rate, and sequence constraints such as 5′ GC bounds, local GC windows, homopolymer limits and restriction-site avoidance.

Six approaches are considered: Greedy optimization, Standard MILP, Constrained MILP, Single-Start Genetic Algorithm (GA), Multi-Start GA and NSGA-II. Greedy makes local codon choices; MILP formulates codon selection mathematically; GA uses selection, crossover and mutation; Multi-Start GA repeats the search from different random seeds; and NSGA-II handles multiple objectives using non-dominated sorting and crowding distance.

The supplied project presentation reports 0% feasibility for Greedy and Standard MILP, and 100% feasibility for Constrained MILP, Single-Start GA, Multi-Start GA and NSGA-II. Reported mean feasible CAI values are 0.9836, 0.8470, 0.8504 and 0.7980 respectively for the latter four methods. NSGA-II reports 628 non-dominated candidates and a mean edit rate of 45.27%. These values are reproduced from the presentation and have not been independently recomputed from raw data.

Contents

1. Introduction

2. Biological Background

3. Problem Statement and Objectives

4. Existing Approaches and Proposed Solution

5. Mathematical Formulation

6. Algorithms and Methodology

7. Tools and Technologies

8. System Architecture and Implementation

9. Experimental Setup and Results

Method	Feasibility	Mean CAI	Mean runtime	Mean edit rate
Greedy	0%	None	0.0002 s	53.76%
Standard MILP	0%	None	0.0793 s	54.34%
Constrained MILP	100%	0.9836	0.8155 s	54.45%
Single-start GA	100%	0.8470	9.2192 s	55.42%
Multi-start GA	100%	0.8504	9.0842 s	54.29%
NSGA-II	100%	0.7980	15.9572 s	45.27%

Table 1. Reported benchmark values from the supplied project presentation; not independently recomputed.

10. Discussion

11. Assumptions and Limitations

12. Challenges and Future Work

13. Conclusion

References

1. Introduction

1.1 Background. Proteins perform essential roles in cells. Genetic information is encoded in DNA, transcribed into messenger RNA (mRNA), and translated by ribosomes into amino-acid sequences. The genetic code is degenerate: most amino acids can be encoded by multiple codons. Alternative codons encoding the same amino acid are called synonymous codons.

1.2 Motivation. Organisms show codon-usage bias, meaning synonymous codons are not used at equal frequencies. Codon usage may interact with tRNA availability, translation dynamics and mRNA structure. Synonymous substitutions can be used to redesign a coding sequence for a target host, but high CAI alone does not guarantee increased protein yield.

1.3 Scope. This project focuses on in-silico optimization of coding DNA sequences for E. coli K-12 MG1655. It compares six optimization approaches under sequence-level constraints. Physical wet-lab expression experiments are outside the scope of this computational study.

1.4 Organization. The report introduces biological concepts, formulates the optimization problem, explains the algorithms, documents the workflow, summarizes reported results and discusses limitations and future extensions.

2. Biological Background

2.1 DNA, genes and coding sequences. DNA stores genetic information as a sequence of nucleotides: adenine (A), thymine (T), guanine (G) and cytosine (C). A gene is a DNA region that contributes to a functional RNA or protein product. A protein-coding sequence is read in groups of three nucleotides called codons.

2.2 Codons and synonymous substitutions. A codon is a three-nucleotide unit that specifies an amino acid or a stop signal. The standard genetic code contains 64 codons: 61 sense codons and three stop codons. For example, CGT and CGC both encode arginine in the standard genetic code.

2.3 Codon usage bias. Codon usage bias describes unequal frequencies among synonymous codons in an organism or reference dataset. A preferred codon is one used relatively frequently in the selected reference. The host reference matters: a codon preferred in one organism is not necessarily preferred in another.

2.4 Codon Adaptation Index. CAI summarizes how closely a coding sequence follows a reference set of preferred codons. Relative adaptiveness is the frequency of a codon divided by the maximum frequency among synonymous codons for the same amino acid. CAI is the geometric mean of relative adaptiveness values. A higher CAI indicates closer alignment to reference preferences, not a guaranteed increase in expression.

2.5 GC content and motifs. GC content is the percentage of bases that are G or C. Global GC is measured across a whole sequence; local GC is measured in a specified window. Homopolymers are runs of one repeated nucleotide, such as AAAAAAA. HindIII recognizes AAGCTT. These motifs are constraints when they conflict with the specified cloning or design workflow.

2.6 Biological caution. Expression also depends on the ribosome-binding site, translation initiation, RNA folding, tRNA availability, protein folding and cellular metabolic load. Final designs require sequence validation and laboratory testing before making claims about expression.

3. Problem Statement and Objectives

3.1 Problem statement. Given an original coding DNA sequence and a target-host codon-usage reference, generate a synonymous sequence that preserves the encoded protein while optimizing codon preference and satisfying sequence constraints. Project constraints include global and local GC bounds, a 5′ GC range, homopolymer or repeat limits and avoidance of specified internal restriction sites.

3.2 Objectives. (1) Preserve the original amino-acid sequence. (2) Improve host-relative codon usage as measured by CAI. (3) Satisfy hard sequence constraints. (4) Compare six optimization approaches. (5) Evaluate feasibility, CAI, runtime and edit rate. (6) Explore trade-offs between codon adaptation and sequence modification.

3.3 Expected output. The system takes a coding sequence as input and produces candidate optimized sequences with metrics and validation outcomes. NSGA-II can produce a set of non-dominated candidates rather than one sequence.

4. Existing Approaches and Proposed Solution

4.1 Greedy optimization. Greedy selects the locally most preferred synonymous codon at each amino-acid position. It is fast but can create global GC violations, forbidden motifs or repeated sequences. Local decisions do not guarantee global feasibility.

4.2 Standard MILP. Mixed-Integer Linear Programming represents codon choices using binary variables and optimizes a mathematical objective. If sequence constraints are omitted from the model, the solution may violate them.

4.3 Constrained MILP and evolutionary methods. Constrained MILP adds sequence restrictions. GA explores candidates using selection, crossover and mutation. Multi-Start GA repeats GA with different seeds. NSGA-II preserves a range of candidates trading off multiple objectives.

4.4 Comparison principle. A fair comparison uses common input data, objective definitions, validation rules and transparent runtime accounting. A high-CAI candidate that violates a hard constraint is not valid under the specification.

5. Mathematical Formulation

5.1 Decision variables. Let p denote an amino-acid position and C(Ap) the synonymous codons for amino acid Ap. Define binary x(p,c)=1 if codon c is selected at position p, and 0 otherwise. Exactly one codon must be selected at each position: $\Sigma[c \text{ in } C(Ap)]\ x(p,c)=1$, with $x(p,c) \text{ in } \{0,1\}$.

5.2 CAI objective. Let wc be relative adaptiveness from the target-host reference. Maximizing CAI is equivalent to maximizing the sum of logarithms: maximize $\Sigma p\ \Sigma c\ \log(wc)x(p,c)$. Care is needed for codons with zero frequency; the implementation must define their handling.

5.3 GC constraints. GC fraction is (G+C)/sequence length. Global bounds constrain the entire sequence; sliding-window bounds constrain local regions; a 5′ GC constraint applies to a specified prefix. The precise linearization depends on each codon's contribution to the relevant bases and windows.

5.4 Motif and repeat constraints. Forbidden restriction sites and repeated patterns can be expressed as constraints over overlapping sequence positions. Pattern constraints may be added iteratively as cuts. HindIII recognizes AAGCTT, which is prohibited only if required by the project specification.

5.5 Edit rate and GC deviation. Edit rate measures the fraction of nucleotide positions that differ from the original sequence. GC deviation is the absolute difference between observed GC fraction and a target fraction. Absolute values can be linearized with auxiliary variables in MILP or evaluated directly in evolutionary algorithms.

5.6 Feasibility. A sequence is feasible when it preserves the required protein sequence and satisfies all hard constraints. An independent validator should recompute translation, GC content, window constraints, homopolymer/repeat rules, forbidden motifs and length.

6. Algorithms and Methodology

6.1 Greedy. At each amino-acid position, select the synonymous codon with the highest relative adaptiveness. This is computationally inexpensive, but it does not account naturally for interactions between codon choices at different positions.

6.2 Standard MILP. Use binary codon-selection variables, enforce one codon per position, and maximize log-CAI. A solver optimizes the encoded model; constraints not included in the model are not automatically enforced.

6.3 Constrained MILP. Add global GC, local GC, 5′ GC and specified motif or repeat restrictions. The supplied presentation identifies PuLP and the COIN-OR CBC solver. Pattern constraints may require cuts or iterative updates. Validate the final sequence independently.

6.4 Single-Start GA. Maintain a population of candidate codon sequences. Each generation evaluates fitness, selects parents, applies crossover and mutation, and preserves elite candidates. The presentation reports population 50, 100 generations, tournament selection k=3, two-point synonymous crossover, mutation probability Pm=0.05 and elitism of the top two candidates.

6.5 Multi-Start GA. Run the GA multiple times with different seeds. The presentation lists 42, 123, 456, 789 and 2026. This explores different search trajectories and reduces dependence on one initialization. It is not inherently multi-objective.

6.6 NSGA-II. Evaluate separate objectives, such as maximizing CAI and minimizing GC deviation and edit rate. Combine parent and offspring populations, perform non-dominated sorting, calculate crowding distance and select the next generation by front rank and diversity. The final Pareto front contains non-dominated candidates.

6.7 Non-dominated sorting and crowding distance. Candidate A dominates B if A is at least as good in every objective and strictly better in one. Non-dominated sorting groups candidates into Pareto fronts. Crowding distance estimates isolation in objective space and helps preserve diversity.

6.8 Constraint handling. Evolutionary algorithms do not automatically guarantee valid sequences. The implementation must use synonymous operators and an explicit constraint-handling strategy such as repair, rejection or penalties. Final candidates must pass independent validation.

7. Tools and Technologies

Python can support sequence parsing, metrics, optimization and validation. Biopython can parse FASTA files and check translations. PuLP is a Python modeling interface for linear and mixed-integer optimization; CBC is a mixed-integer solver. GA and NSGA-II can be implemented in custom code or an evolutionary library.

The project context identifies ten genes: GFP, firefly luciferase, HBB, INS, LYZ, ACTB, TP53, lacZ, groL and sodA. The presentation describes FASTA inputs and CSV outputs. Accession identifiers and exact file contents should be verified against actual project data before submission.

8. System Architecture and Implementation

8.1 Workflow. Read a coding sequence in FASTA format and a target-host codon-frequency reference. Check valid bases, coding length, reading frame and translation. Run an optimizer to generate candidates. Calculate CAI, GC statistics and edit rate. Apply an independent validator, then export metrics and outcomes to CSV.

8.2 Modules. (1) Input parser; (2) host reference-data module; (3) optimizer modules; (4) metrics module; (5) constraint validator; (6) benchmark driver; (7) reporting and plotting; (8) Pareto-candidate selection.

8.3 Reproducibility. Record random seeds, software versions, solver settings, stopping criteria and data identifiers. Define evaluation budgets and state whether runtime includes preprocessing and validation.

9. Experimental Setup and Results

9.1 Benchmark design. The presentation reports a benchmark of ten genes. Methods are compared using feasibility, mean feasible CAI, runtime and edit rate. The values below are reproduced from the presentation and should be verified against raw CSV logs before formal publication.

10. Discussion

10.1 Feasibility versus quality. A candidate that violates a required sequence constraint is invalid even if its CAI is high. The reported Greedy and Standard MILP results highlight the importance of constraint handling.

10.2 Exact optimization versus population search. MILP can provide an optimum for the encoded model if the solver completes and proves optimality. GA and NSGA-II use stochastic search and depend on encoding, operators, parameter settings and random seeds.

10.3 Single versus multiple objectives. Single-objective GA typically combines properties into one fitness score. Multi-Start GA repeats that search. NSGA-II returns a Pareto front, enabling selection between high CAI and lower edit rate.

10.4 Biological interpretation. CAI, GC and motif constraints are computational proxies. They do not directly establish mRNA stability, translation rate, protein folding or final yield. Experimental validation is required for expression claims.

11. Assumptions and Limitations

1. The benchmark uses E. coli K-12 MG1655; conclusions may not transfer to other hosts.

2. Ten genes are a small benchmark, not a genome-wide evaluation.

3. CAI and sequence metrics do not guarantee expression improvement.

4. Results depend on thresholds, codon-frequency data and algorithm parameters.

5. Evolutionary algorithms are stochastic; repeated trials and recorded seeds are important.

6. Hard constraints require independent validation.

7. Wet-lab experiments are needed to test expression, folding and yield.

8. Runtime comparisons depend on hardware, software versions and timing methodology.

12. Challenges and Future Work

Challenges include balancing codon preference against constraints, representing overlapping motifs, avoiding premature convergence, choosing penalty or repair strategies, and ensuring fair comparisons. Multi-objective methods also require a principled way to choose a final candidate from the Pareto front.

Future work can include RNA secondary-structure analysis near the translation initiation region, broader host and gene benchmarks, additional random seeds, improved constraint-aware genetic operators, and wet-lab comparisons between original and optimized sequences. Ribosome profiling or cell-free expression assays could provide evidence about translation and protein production. These are proposed extensions, not completed experiments.

13. Conclusion

This project formulates codon optimization as a constrained sequence-design problem and compares Greedy, Standard MILP, Constrained MILP, Single-Start GA, Multi-Start GA and NSGA-II. Greedy makes local choices; MILP optimizes an encoded mathematical model; GA uses population-based evolution; Multi-Start GA repeats the search; and NSGA-II preserves non-dominated trade-offs.

The supplied presentation reports 100% feasibility for constrained and evolutionary approaches and 0% for Greedy and Standard MILP under its validator. Constrained MILP had the highest reported mean feasible CAI, while NSGA-II returned a reported Pareto set of 628 candidates with a lower mean edit rate. These findings support explicit constraint handling and multi-objective analysis, but apply only to the reported benchmark and require raw-data verification and experimental validation before biological expression claims.

References

[1] P. M. Sharp and W.-H. Li, “The codon Adaptation Index—a measure of directional synonymous codon usage bias, and its potential applications,” Nucleic Acids Research, vol. 15, no. 3, pp. 1281–1295, 1987.

[2] NCBI, “The Genetic Codes,” National Center for Biotechnology Information. https://www.ncbi.nlm.nih.gov/Taxonomy/Utils/wprintgc.cgi

[3] PuLP documentation, “PuLP: an LP modeler in Python.” https://coin-or.github.io/pulp/

[4] COIN-OR, “CBC project documentation.” https://github.com/coin-or/Cbc

[5] K. Deb, A. Pratap, S. Agarwal and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” IEEE Transactions on Evolutionary Computation, vol. 6, no. 2, pp. 182–197, 2002.

[6] D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley, 1989.

[7] Supplied project presentation, “Constraint-Aware Codon Optimization Using Greedy, MILP, Genetic Algorithms and NSGA-II.” Results reproduced as reported; raw benchmark data were not included for independent verification.